

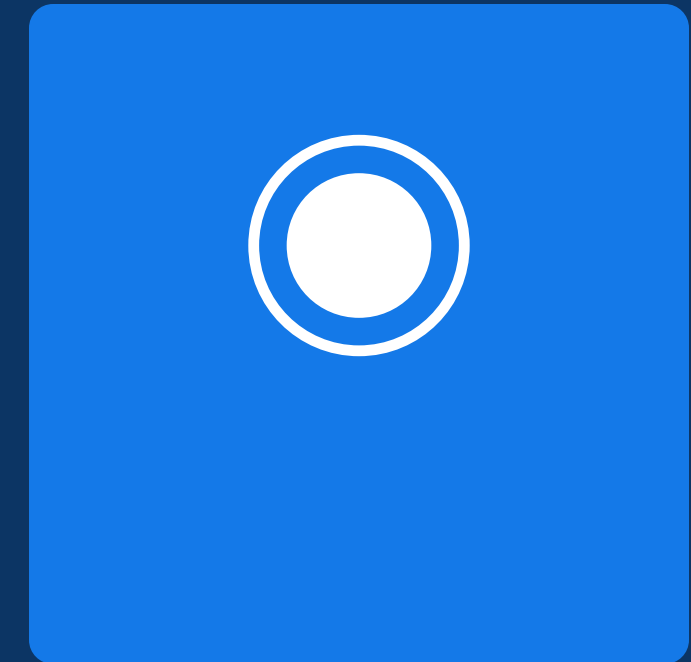
NewLinkWireless

Predicción de cancelación de clientes

Proyecto final de Ciencia de Datos II

Allan Guillermo Bennett Torres
Carlos Salvador López Chinchilla
Darleny Abigail Gómez Bulnes

CEUTEC San Pedro Sula | Sección 77



El CRM ya reúne las señales necesarias

NewLinkWireless administra la operación diaria de una empresa proveedora de internet.

01 **Clientes y servicios**
Contratos, planes y antigüedad

02 **Facturación y pagos**
Facturas, saldos y días de atraso

03 **Soporte técnico**
Tickets, prioridad y resolución

04 **Interacciones**
Seguimiento e historial del cliente

SQL Server



CSV consolidado



Análisis y Power BI

La empresa reacciona cuando el cliente ya canceló

Situación actual

Los datos se revisan de forma individual y la empresa pierde margen para intervenir antes de la cancelación.

Pregunta central

¿Podemos estimar el riesgo con datos del CRM?

Señales previas

- Más tickets de soporte
- Facturas vencidas
- Atrasos frecuentes
- Menor antigüedad
- Problemas técnicos pendientes

¿Qué nos dicen los datos sobre la cancelación?

Principales hallazgos del análisis exploratorio (EDA)

Cancelación	Antigüedad	Tickets de soporte	Atrasos
201%	19.85 meses	4.72 tickets	13.6 días
201 de 1,000 servicios cancelaron	Canceló vs. 32.05 meses no canceló	Canceló vs. 2.17 no canceló	Canceló vs. 6 días no canceló

Preparación y validación del dataset

```
print("Valores nulos por columna:")
print(df.isnull().sum())

print("\nTotal de valores nulos:", df.isnull().sum().sum())
```

Valores nulos por columna:

```
id_servicio      0
tipo_cliente     0
tipo_servicio    0
plan             0
velocidad_bajada_mbps  0
velocidad_subida_mbps  0
precio_contratado  0
antiguedad_meses  0
cantidad_equipos  0
cantidad_tickets  0
tickets_abiertos  0
tickets_criticos  0
promedio_dias_resolucion  0
cantidad_facturas  0
facturas_vencidas  0
monto_facturado  0
monto_pagado     0
saldo_pendiente  0
dias_atraso_promedio  0
cantidad_interacciones  0
cancelo_servicio  0
dtype: int64
```

Total de valores nulos: 0

```
duplicados = df.duplicated().sum()
```

```
print("Registros duplicados:", duplicados)
```

Registros duplicados: 0

```
inconsistencias_tickets = df[
    (df["tickets_abiertos"] > df["cantidad_tickets"]) |
    (df["tickets_criticos"] > df["cantidad_tickets"])
]

inconsistencias_tickets[
    ["id_servicio", "cantidad_tickets", "tickets_abiertos", "tickets_criticos"]
]
```

	id_servicio	cantidad_tickets	tickets_abiertos	tickets_criticos
106	107	0	0	1
335	336	0	1	0
717	718	5	3	6

```
df["tickets_abiertos"] = df[
    ["tickets_abiertos", "cantidad_tickets"]
].min(axis=1)

df["tickets_criticos"] = df[
    ["tickets_criticos", "cantidad_tickets"]
].min(axis=1)
```

```
print("Inconsistencias corregidas.")
```

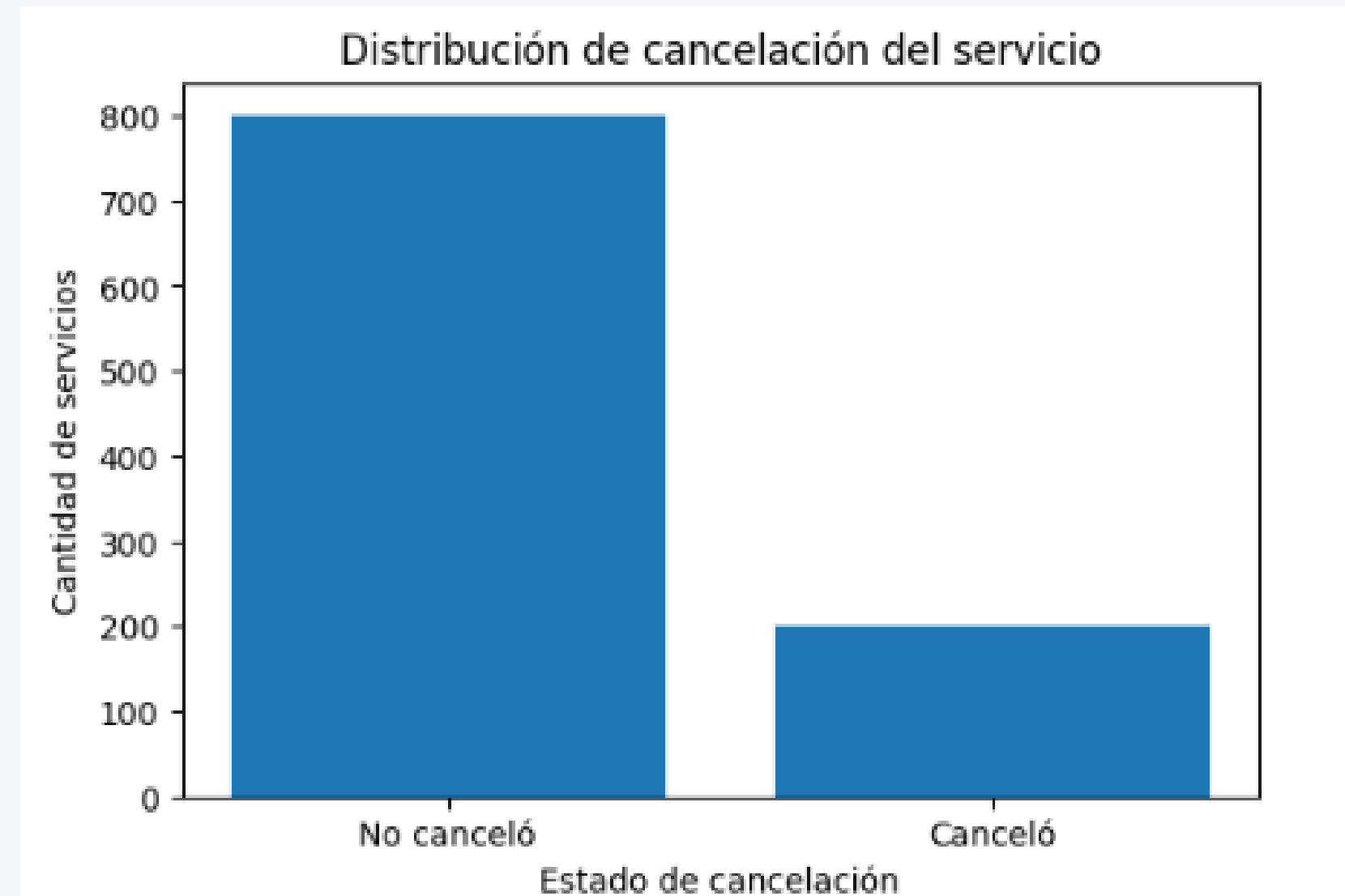
Inconsistencias corregidas.

```
print(
    "Tickets abiertos mayores al total:",
    (df["tickets_abiertos"] > df["cantidad_tickets"]).sum()
)

print(
    "Tickets críticos mayores al total:",
    (df["tickets_criticos"] > df["cantidad_tickets"]).sum()
)
```

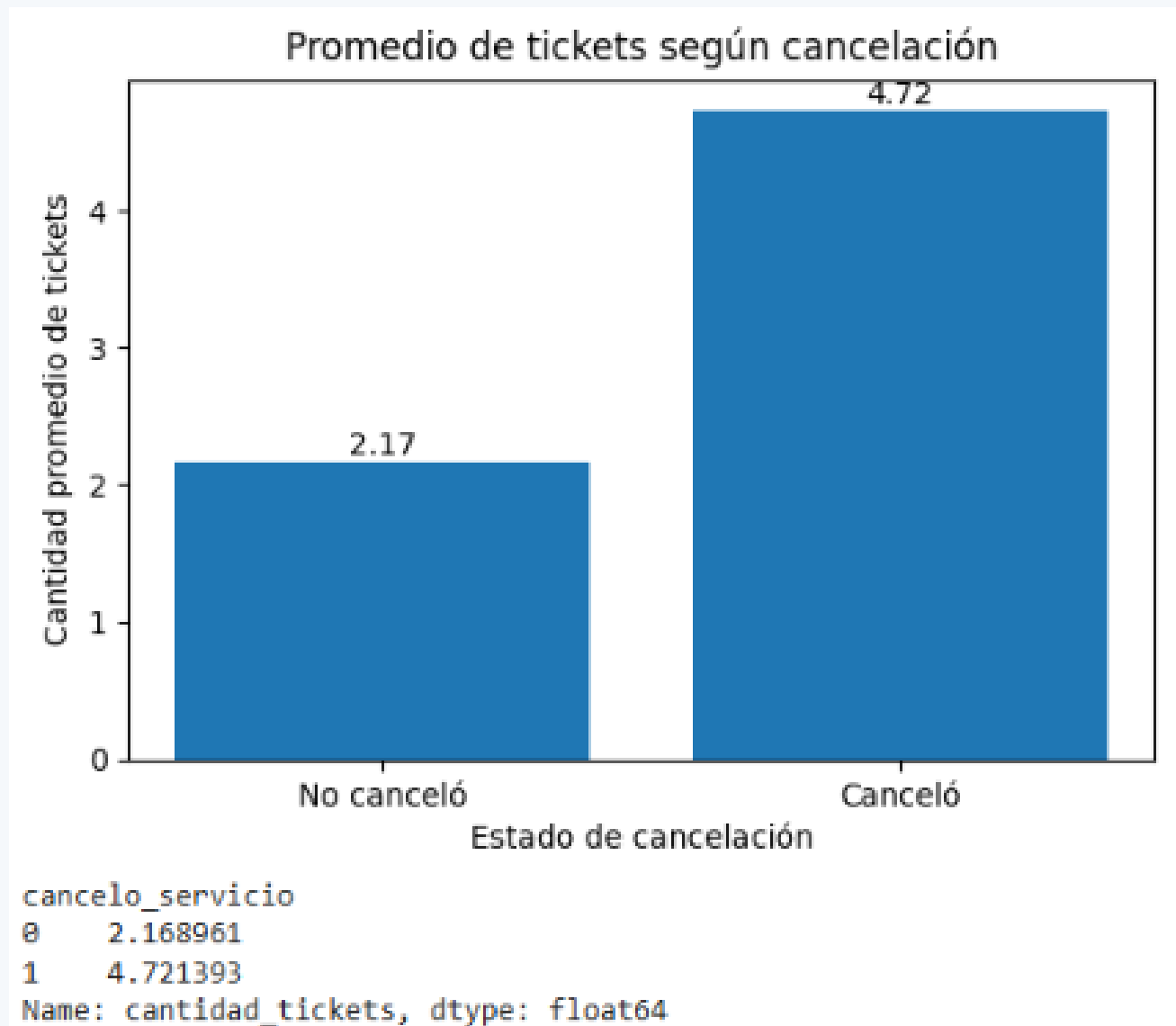
Tickets abiertos mayores al total: 0
Tickets críticos mayores al total: 0

La antigüedad también cuenta una historia



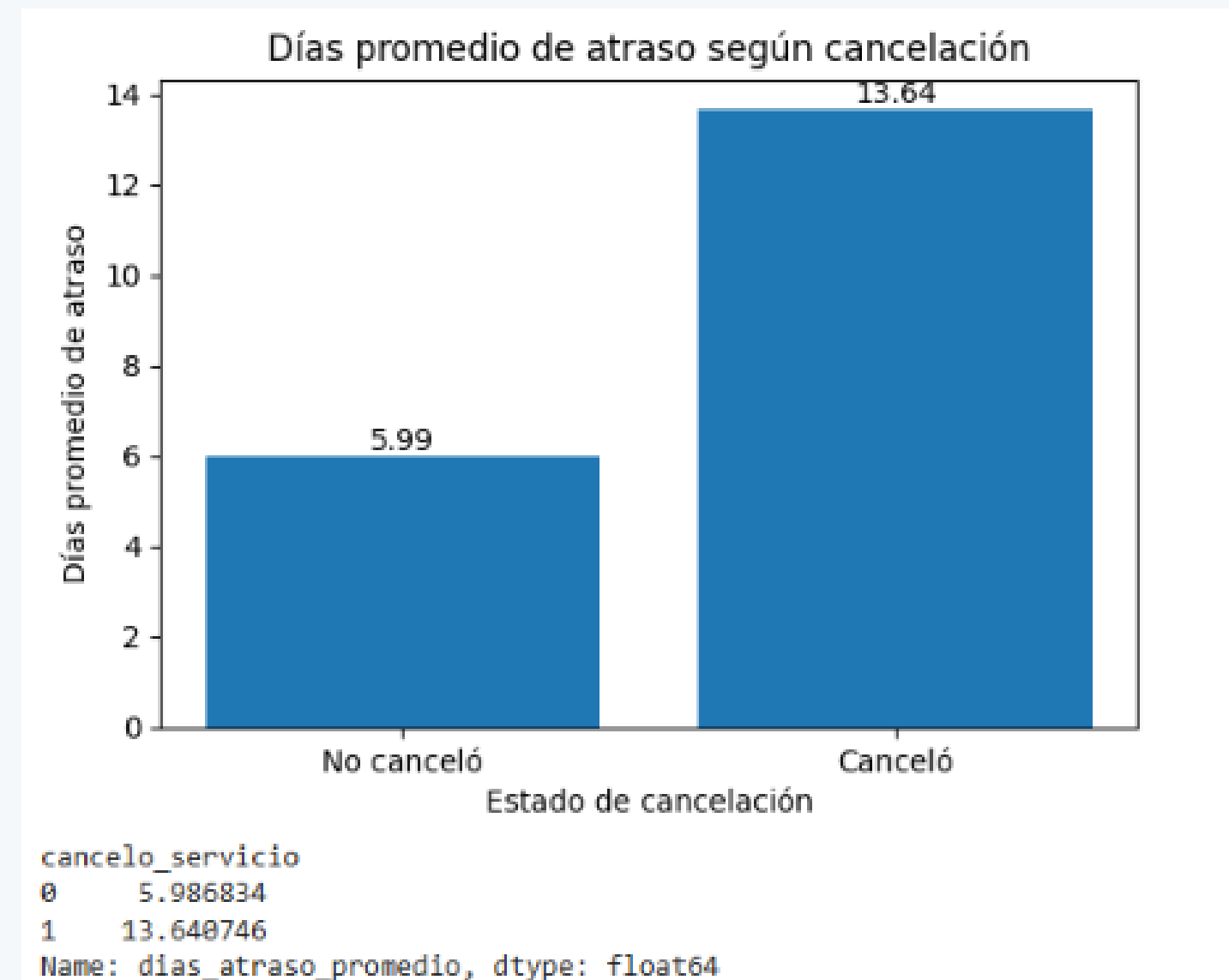
Los servicios que no cancelaron tenían una antigüedad promedio de aproximadamente 32 meses, mientras que los que cancelaron tenían alrededor de 20 meses.

Más del doble de tickets de soporte



Los servicios cancelados registraron más del doble de tickets de soporte en promedio.

El comportamiento de pago también muestra diferencias



Los servicios cancelados presentaron mayores atrasos promedio en sus pagos.

Preparación de los datos para Machine Learning

Para preparar los datos, se definió `cancelo_servicio` como la variable objetivo y se realizaron las transformaciones necesarias para que la información pudiera ser utilizada por los modelos. Después, los datos se dividieron en conjuntos de entrenamiento y prueba para evaluar el rendimiento de los modelos.

```
X = df.drop(columns=["cancelo_servicio", "id_servicio"])
y = df["cancelo_servicio"]

print("Dimensiones de X:", X.shape)
print("Dimensiones de y:", y.shape)

print("\nDistribución de la variable objetivo:")
print(y.value_counts())

print("\nPorcentaje:")
print(y.value_counts(normalize=True) * 100)
```



```
Dimensiones de X: (1000, 19)
Dimensiones de y: (1000,)

Distribución de la variable objetivo:
cancelo_servicio
0      799
1      201
Name: count, dtype: int64

Porcentaje:
cancelo_servicio
0      79.9
1      20.1
Name: proportion, dtype: float64
```

Preparación de los datos para Machine Learning

Se identificaron las variables numéricas y categóricas del dataset para aplicar el tratamiento correspondiente antes de entrenar los modelos.

Las variables categóricas requieren una transformación, mientras que las numéricas pueden utilizarse directamente.

```
columnas_categoricas = [  
    "tipo_cliente",  
    "tipo_servicio",  
    "plan"  
]  
  
columnas_numericas = [  
    columna for columna in X.columns  
    if columna not in columnas_categoricas  
]  
  
print("Variables categóricas:")  
print(columnas_categoricas)  
  
print("\nVariables numéricas:")  
print(columnas_numericas)
```

```
Variables categóricas:  
['tipo_cliente', 'tipo_servicio', 'plan']
```

```
Variables numéricas:  
['velocidad_bajada_mbps', 'velocidad_subida_mbps', 'precio_contratado', '...
```

Preparación de los datos para Machine Learning

Las variables categóricas se transformaron mediante One-Hot Encoding para convertir sus categorías en valores numéricos que puedan ser utilizados por los modelos.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

preprocesador = ColumnTransformer(
    transformers=[
        (
            "categoricas",
            OneHotEncoder(handle_unknown="ignore"),
            columnas_categoricas
        ),
        (
            "numericas",
            StandardScaler(),
            columnas_numericas
        )
    ]
)

print("Preprocesador preparado correctamente.")
```

Preprocesador preparado correctamente.

Preparación de los datos para Machine Learning

El dataset se dividió en 80% para entrenamiento y 20% para prueba, manteniendo una proporción similar de clientes activos y cancelados en ambos grupos.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Datos de entrenamiento:", X_train.shape)
print("Datos de prueba:", X_test.shape)

print("\nDistribución en entrenamiento:")
print(y_train.value_counts())

print("\nDistribución en prueba:")
print(y_test.value_counts())
```

```
Datos de entrenamiento: (800, 19)
Datos de prueba: (200, 19)
```

```
Distribución en entrenamiento:
cancelo_servicio
0    639
1    161
Name: count, dtype: int64
```

```
Distribución en prueba:
cancelo_servicio
0    160
1     40
Name: count, dtype: int64
```

Entrenamiento de modelos de clasificación

La Regresión Logística se utilizó como primer modelo para predecir si un cliente puede cancelar o no el servicio.

El modelo se evaluó con el conjunto de prueba utilizando las métricas Accuracy, Precision, Recall y F1-Score para medir su desempeño.

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

modelo_logistico = Pipeline([
    ("preprocesamiento", preprocesador),
    ("modelo", LogisticRegression(max_iter=1000))
])

modelo_logistico.fit(X_train, y_train)

print("Modelo de Regresión Logística entrenado correctamente.")
```

Modelo de Regresión Logística entrenado correctamente.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred_logistico = modelo_logistico.predict(X_test)

accuracy_logistico = accuracy_score(y_test, y_pred_logistico)
precision_logistico = precision_score(y_test, y_pred_logistico)
recall_logistico = recall_score(y_test, y_pred_logistico)
f1_logistico = f1_score(y_test, y_pred_logistico)

print("Resultados - Regresión Logística")
print("-----")
print(f"Accuracy: {accuracy_logistico:.4f}")
print(f"Precision: {precision_logistico:.4f}")
print(f"Recall: {recall_logistico:.4f}")
print(f"F1-Score: {f1_logistico:.4f}")
```

Resultados - Regresión Logística

Accuracy: 0.8300
Precision: 0.6071
Recall: 0.4250
F1-Score: 0.5000

Entrenamiento de modelos de clasificación

El Árbol de Decisión se utilizó como segundo modelo de clasificación, limitando su profundidad para evitar un ajuste excesivo a los datos.

El Árbol de Decisión se evaluó con las mismas métricas utilizadas anteriormente para poder comparar su rendimiento con la Regresión Logística.

```
from sklearn.tree import DecisionTreeClassifier

modelo_arbol = Pipeline([
    ("preprocesamiento", preprocesador),
    ("modelo", DecisionTreeClassifier(
        random_state=42,
        max_depth=5
    ))
])

modelo_arbol.fit(X_train, y_train)

print("Modelo de Árbol de Decisión entrenado correctamente.")
```

Modelo de Árbol de Decisión entrenado correctamente.

```
y_pred_arbol = modelo_arbol.predict(X_test)

accuracy_arbol = accuracy_score(y_test, y_pred_arbol)
precision_arbol = precision_score(y_test, y_pred_arbol)
recall_arbol = recall_score(y_test, y_pred_arbol)
f1_arbol = f1_score(y_test, y_pred_arbol)

print("Resultados - Árbol de Decisión")
print("-----")
print(f"Accuracy:  {accuracy_arbol:.4f}")
print(f"Precision: {precision_arbol:.4f}")
print(f"Recall:    {recall_arbol:.4f}")
print(f"F1-Score:   {f1_arbol:.4f}")
```

Resultados - Árbol de Decisión

Accuracy: 0.8250

Precision: 0.5610

Recall: 0.5750

F1-Score: 0.5679

Entrenamiento de modelos de clasificación

La matriz de confusión se utilizó para observar con mayor detalle los aciertos y errores del modelo, incluyendo falsos positivos y falsos negativos.

El modelo identificó correctamente 142 servicios activos y 23 cancelados. También presentó 18 falsos positivos y 17 falsos negativos.

Estos resultados muestran que el Árbol de Decisión detectó una parte importante de las cancelaciones, aunque todavía presentó errores en algunas predicciones.

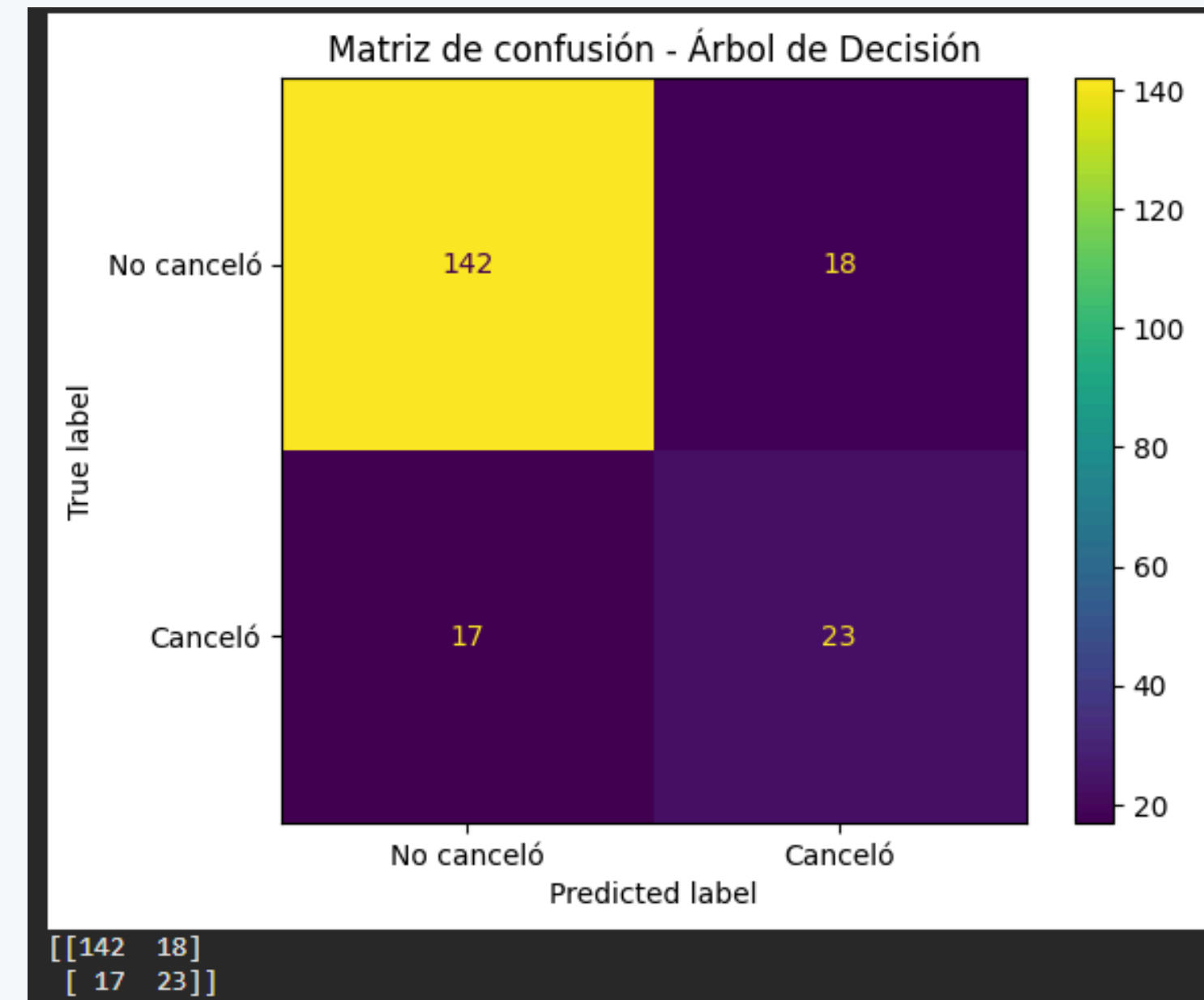
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Crear matriz de confusión
matriz = confusion_matrix(y_test, y_pred_arbol)

# Mostrar matriz
disp = ConfusionMatrixDisplay(
    confusion_matrix=matriz,
    display_labels=["No canceló", "Canceló"]
)

disp.plot()
plt.title("Matriz de confusión - Árbol de Decisión")
plt.show()

print(matriz)
```



Entrenamiento de modelos de clasificación

La matriz de confusión se utilizó para observar con mayor detalle los aciertos y errores del modelo, incluyendo falsos positivos y falsos negativos.

El modelo identificó correctamente 142 servicios activos y 23 cancelados. También presentó 18 falsos positivos y 17 falsos negativos.

Estos resultados muestran que el Árbol de Decisión detectó una parte importante de las cancelaciones, aunque todavía presentó errores en algunas predicciones.

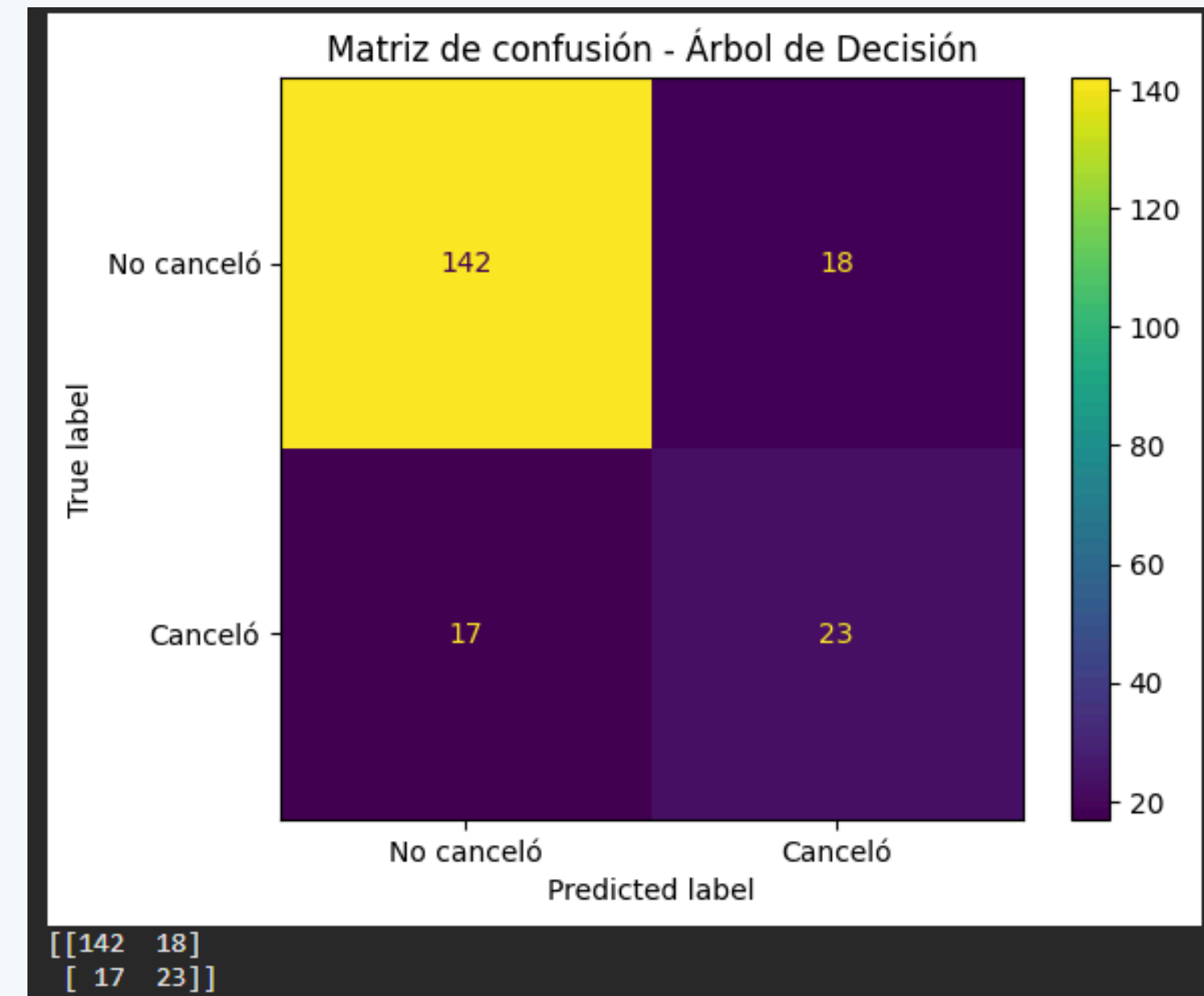
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Crear matriz de confusión
matriz = confusion_matrix(y_test, y_pred_arbol)

# Mostrar matriz
disp = ConfusionMatrixDisplay(
    confusion_matrix=matriz,
    display_labels=["No canceló", "Canceló"]
)

disp.plot()
plt.title("Matriz de confusión - Árbol de Decisión")
plt.show()

print(matriz)
```



Entrenamiento de modelos de clasificación

Random Forest se utilizó como tercer modelo de clasificación, combinando varios árboles de decisión para obtener predicciones más estables.

```
y_pred_rf = modelo_rf.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

print("Resultados - Random Forest")
print("-----")
print(f"Accuracy:  {accuracy_rf:.4f}")
print(f"Precision: {precision_rf:.4f}")
print(f"Recall:    {recall_rf:.4f}")
print(f"F1-Score:   {f1_rf:.4f}")

Resultados - Random Forest
-----
Accuracy:  0.8350
Precision: 0.6129
Recall:    0.4750
F1-Score:  0.5352
```

```
from sklearn.ensemble import RandomForestClassifier

modelo_rf = Pipeline([
    ("preprocesamiento", preprocesador),
    ("modelo", RandomForestClassifier(
        n_estimators=200,
        random_state=42
    ))
])

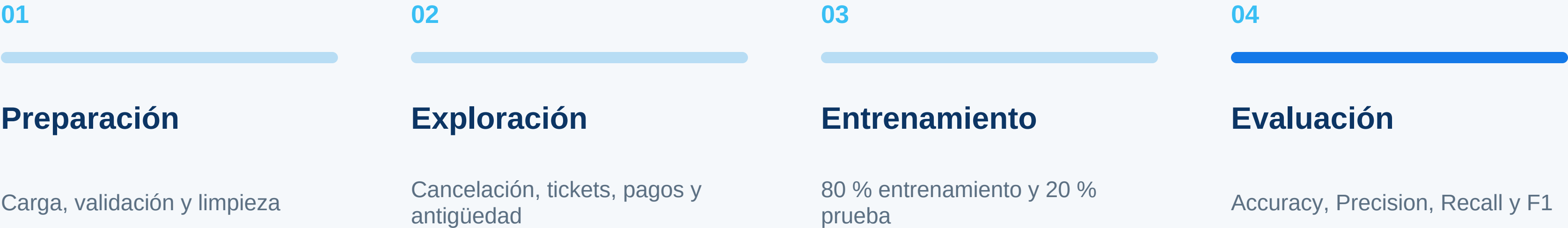
modelo_rf.fit(X_train, y_train)

print("Modelo Random Forest entrenado correctamente.")

Modelo Random Forest entrenado correctamente.
```

Random Forest se evaluó con las mismas métricas utilizadas en los modelos anteriores para comparar su rendimiento.

Del archivo CSV al modelo seleccionado



Modelos comparados

Regresión Logística

Árbol de Decisión

Random Forest

Modelo final

Árbol de Decisión

	Modelo	Accuracy	Precision	Recall	F1-Score
0	Regresión Logística	0.830	0.607143	0.425	0.500000
1	Árbol de Decisión	0.825	0.560976	0.575	0.567901
2	Random Forest	0.835	0.612903	0.475	0.535211

20.10 % de los servicios fueron cancelados

1,000

Servicios analizados

201

Servicios cancelados

82.50 %

Accuracy

57.50 %

Recall

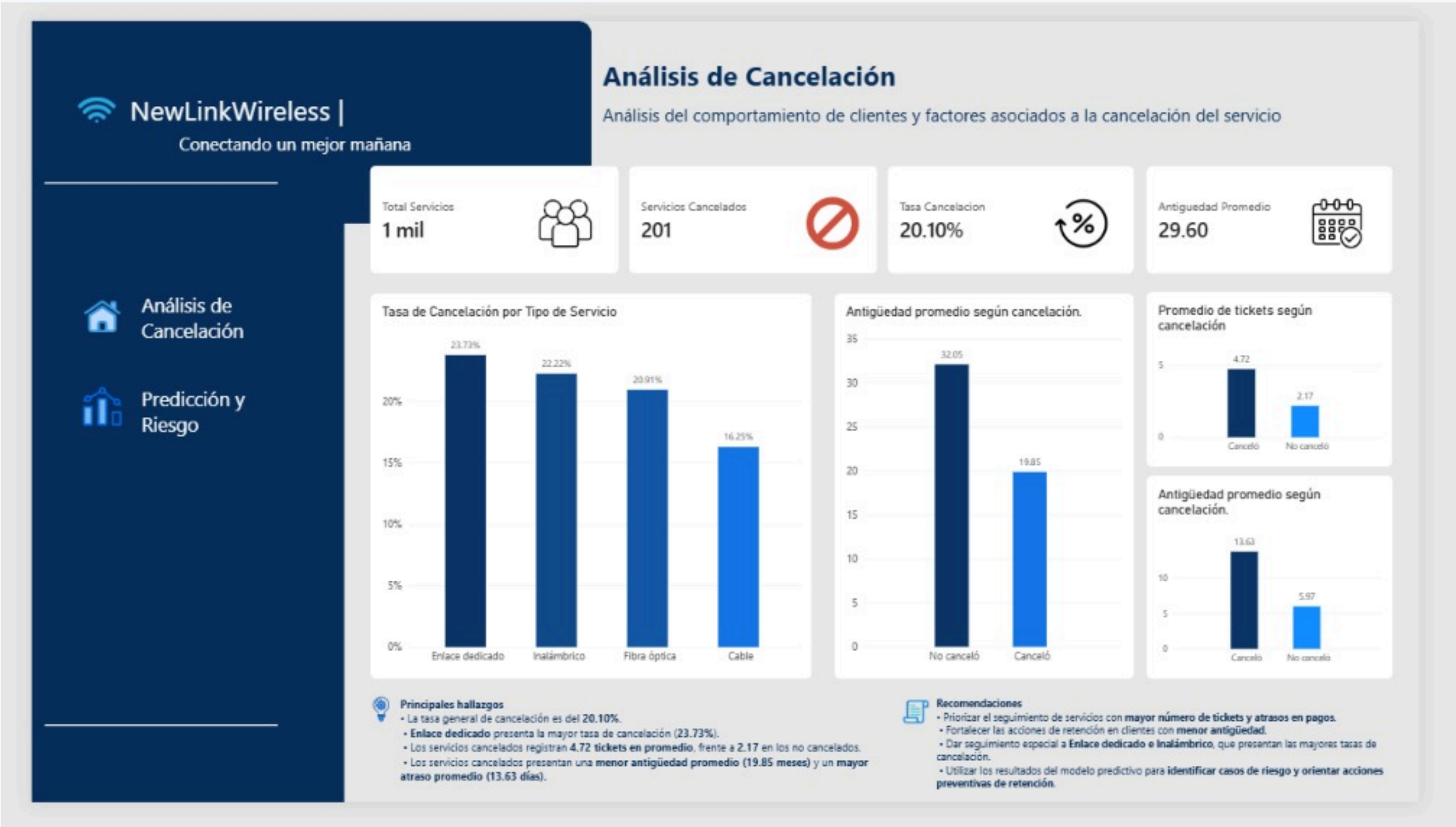
Distribución por nivel de riesgo



Matriz de confusión

142	No cancelaciones correctas
23	Cancelaciones detectadas
18	Falsas alertas
17	Cancelaciones no detectadas

Los cancelados acumulan más tickets y atraso



107 servicios concentran la atención inmediata



Los datos del CRM permiten actuar antes

01

Señales útiles

Tickets, atrasos, facturas vencidas y antigüedad muestran asociaciones con la cancelación.

02

Priorización

El modelo convierte el historial del CRM en niveles de riesgo que facilitan el seguimiento.

03

Decisión asistida

La predicción orienta al personal, pero no sustituye la revisión individual del cliente.

Valor agregado: enfocar los recursos de soporte y retención en los casos con mayor riesgo.

Siguiente etapa para NewLinkWireless

1 Prioridad inmediata

Revisar los 107 servicios con riesgo alto y asignar seguimiento.

2 Atención preventiva

Contactar clientes con varios tickets, atrasos o poca antigüedad.

3 Integración

Conectar las predicciones con el CRM y generar alertas automáticas.

4 Mejora del modelo

Incorporar datos reales, ajustar el umbral y evaluar el modelo periódicamente.

La meta es intervenir temprano y medir si las acciones de retención reducen las cancelaciones.